

AI SOFTWARE DEVELOPMENT INTERNSHIP

EMAN MASOOD (BSSE)

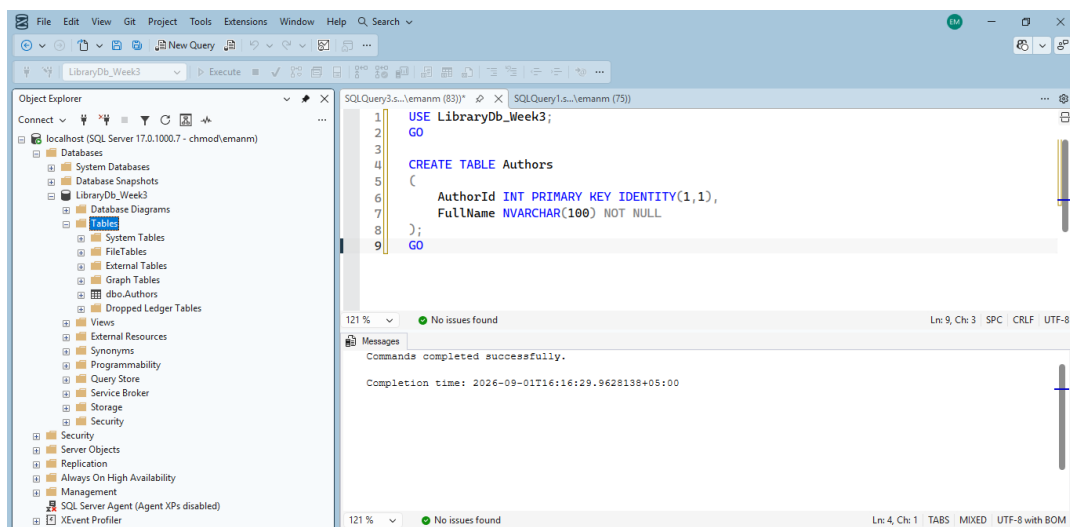
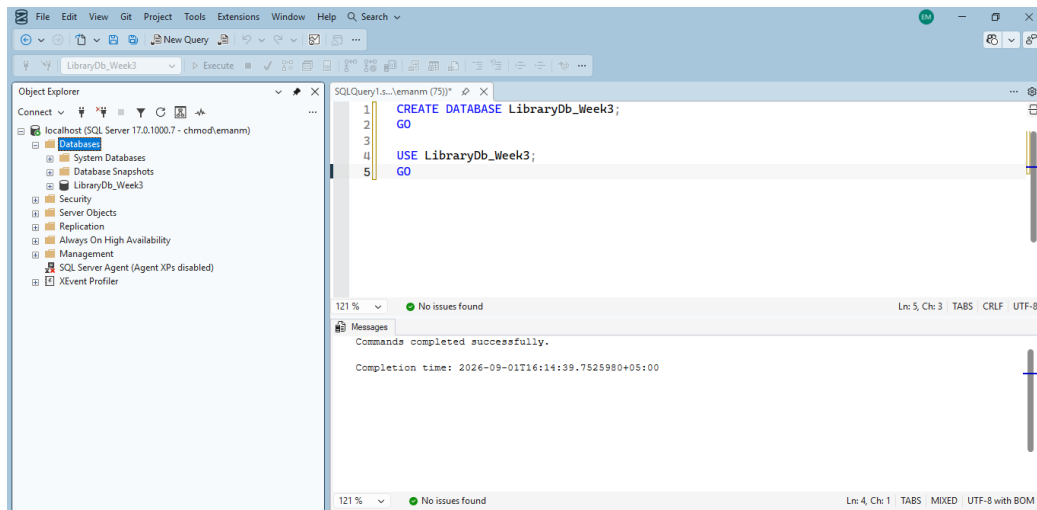
Week 3: SQL Server & Entity Framework Core • API/Angular
Integration Git & GitHub Workflow • AI & Python Foundations

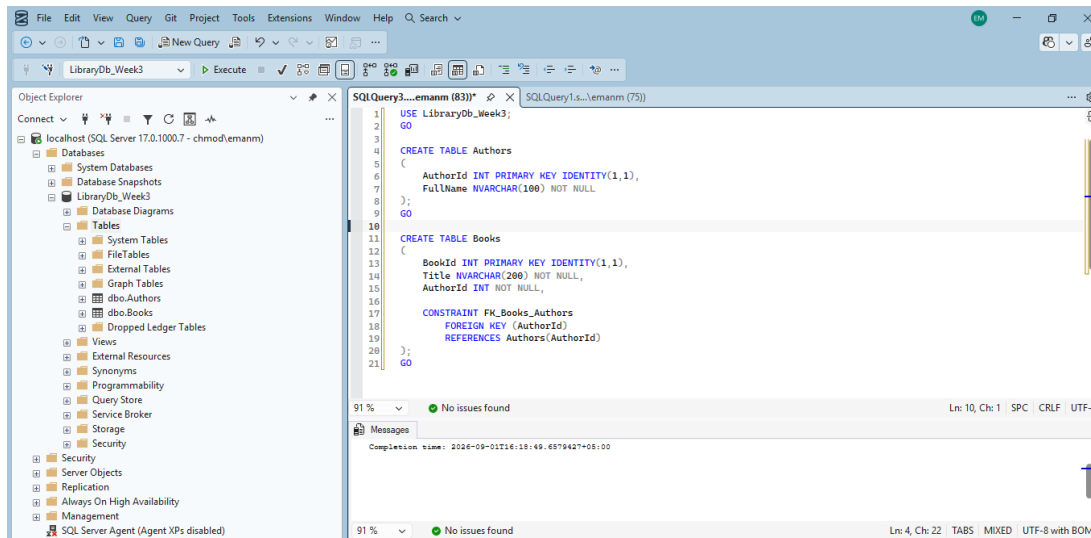
PART A — RELATIONAL DATABASE DESIGN (BEYOND BASIC TABLES)

Practice Tasks

1. Create an Authors table (AuthorId PK, FullName) and a Books table (BookId PK, Title, AuthorId FK referencing Authors).

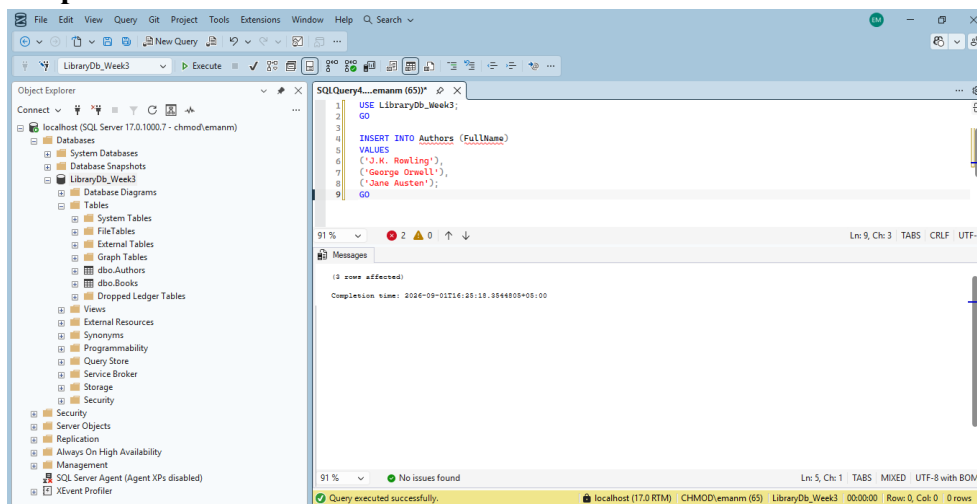
Output:





2. Insert 3 authors and 6 books split between them

Output:



SQL Server Enterprise Edition (64-bit) - SQL Server 17.0.1000.7 - chmod\emanm

LibraryDb_Week3

Execute

Object Explorer

Connect

localhost (SQL Server 17.0.1000.7 - chmod\emanm)

Databases

System Databases

Database Snapshots

LibraryDb_Week3

Database Diagrams

Tables

System Tables

FileTables

External Tables

Graph Tables

dbo.Authors

dbo.Books

Dropped Ledger Tables

Views

External Resources

Synonyms

Programmability

Query Store

Service Broker

SQLQuery5....emanm (67)*

SQLQuery4.s....\emanm (65)*

1 | SELECT * FROM Authors;

91 %

Results

Messages

	AuthorId	FullName
1	1	J.K. Rowling
2	2	George Orwell
3	3	Jane Austen

SQL Server Enterprise Edition (64-bit) - SQL Server 17.0.1000.7 - chmod\emanm

LibraryDb_Week3

Execute

Object Explorer

Connect

localhost (SQL Server 17.0.1000.7 - chmod\emanm)

Databases

System Databases

Database Snapshots

LibraryDb_Week3

Database Diagrams

Tables

System Tables

FileTables

External Tables

Graph Tables

dbo.Authors

dbo.Books

Dropped Ledger Tables

Views

External Resources

Synonyms

Programmability

Query Store

Service Broker

Storage

Security

SQLQuery6....emanm (76)*

SQLQuery5.s....\emanm (67)*

task1a.sql - ...D\emanm (65))

```
1 | INSERT INTO Books (Title, AuthorId)
2 | VALUES
3 | ('Harry Potter and the Philosopher's Stone', 1),
4 | ('Harry Potter and the Chamber of Secrets', 1),
5 | ('1984', 2),
6 | ('Animal Farm', 2),
7 | ('Pride and Prejudice', 3),
8 | ('Sense and Sensibility', 3);
9 | GO
10 |
11 |
```

91 %

Messages

(6 rows affected)

Completion time: 2026-09-01T16:28:54.8082866+05:00

The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'localhost (SQL Server 17.0.1000.7 - chmod\emanm)', including 'LibraryDb_Week3'. The query window on the right contains the following SQL statement:

```
1 SELECT * FROM Books;
```

The results pane shows a table with 6 rows and 3 columns: BookId, Title, and AuthorId.

BookId	Title	AuthorId
1	Harry Potter and the Philosopher's Stone	1
2	Harry Potter and the Chamber of Secrets	1
3	1984	2
4	Animal Farm	2
5	Pride and Prejudice	3
6	Sense and Sensibility	3

3. Write a JOIN query that lists every book together with its author's name in one result set.

Output:

The screenshot shows the SQL Server Enterprise Manager interface. The query window on the right contains the following SQL statement:

```
1 USE LibraryDb_Week3;
2 GO
3
4 SELECT
5     Books.Title AS Book,
6     Authors.FullName AS Authors
7 FROM Books
8 INNER JOIN Authors
9     ON Books.AuthorId = Authors.AuthorId
10 ORDER BY Authors.FullName;
```

The results pane shows a table with 6 rows and 2 columns: Book and Author.

Book	Author
1984	George Orwell
Animal Farm	George Orwell
Harry Potter and the Philosopher's Stone	J.K. Rowling
Harry Potter and the Chamber of Secrets	J.K. Rowling
Pride and Prejudice	Jane Austen
Sense and Sensibility	Jane Austen

The status bar at the bottom indicates: "Query executed successfully. localhost (17.0 RTM) | CHMOD\emanm (76) | LibraryDb_Week3 | 00:00:00 | Row: 1, Col: 1 | 6 rows".

4. Create a Categories table and a link table BookCategories (BookId, CategoryId) to represent many-to-many, then insert sample rows connecting books to more than one category.

Output:

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure for 'LibraryDb_Week3', including tables like 'dbo.Authors', 'dbo.Books', and 'dbo.Categories'. The main window shows a SQL query window with the following code:

```
1 USE LibraryDb_Week3;
2 GO
3
4 CREATE TABLE Categories
5 (
6     CategoryId INT PRIMARY KEY IDENTITY(1,1),
7     CategoryName NVARCHAR(100) NOT NULL
8 );
9 GO
```

The Messages pane at the bottom indicates that the commands completed successfully and provides the completion time: 2026-09-01T16:48:59.0370647+05:00.

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the Object Explorer displays the database structure for 'LibraryDb_Week3', including tables like 'dbo.Authors', 'dbo.Books', and 'dbo.Categories'. The main window shows a SQL query window with the following code:

```
9
10 CONSTRAINT PK_BookCategories
11     PRIMARY KEY (BookId, CategoryId),
12
13 CONSTRAINT FK_BookCategories_Books
14     FOREIGN KEY (BookId)
15     REFERENCES Books(BookId),
16
17 CONSTRAINT FK_BookCategories_Categories
18     FOREIGN KEY (CategoryId)
19     REFERENCES Categories(CategoryId)
20 );
21 GO
```

The Messages pane at the bottom indicates that the commands completed successfully and provides the completion time: 2026-09-01T16:51:07.0831625+05:00.

SQL Server Enterprise Edition (64-bit) - CHMOD\emanm (82)

File Edit View Query Git Project Tools Extensions Window Help Search

LibraryDb_Week3 Execute

Object Explorer

Connect localhost (SQL Server 17.0.1000.7 - chmod\emanm)

Databases

System Databases

Database Snapshots

LibraryDb_Week3

Database Diagrams

Tables

System Tables

FileTables

External Tables

Graph Tables

dbo.Authors

dbo.BookCategories

dbo.Books

dbo.Categories

Dropped Ledger Tables

Views

External Resources

Synonyms

Programmability

Query Store

Service Broker

Storage

Security

Server Objects

Replication

Always On High Availability

Management

SQL Server Agent (Agent XPs disabled)

SQLQuery12...manm (82)*

task4b.sql - ...D\emanm (56)

task4a.sql - ...D\emanm (76)

```
1 USE LibraryDb_Week3;
2 GO
3
4 INSERT INTO Categories (CategoryName)
5 VALUES
6 ('Fantasy'),
7 ('Fiction'),
8 ('Classic'),
9 ('Dystopian');
10 GO
11
12 SELECT * FROM Categories;
```

121 % 3 0

Results Messages

	CategoryId	CategoryName
1	1	Fantasy
2	2	Fiction
3	3	Classic
4	4	Dystopian

Query executed successfully. localhost (17.0 RTM) CHMOD\emanm (82) LibraryDb_V

SQL Server Enterprise Edition (64-bit) - CHMOD\emanm (78)

File Edit View Query Git Project Tools Extensions Window Help Search

LibraryDb_Week3 Execute

Object Explorer

Connect localhost (SQL Server 17.0.1000.7 - chmod\emanm)

Databases

System Databases

Database Snapshots

LibraryDb_Week3

Database Diagrams

Tables

System Tables

FileTables

External Tables

Graph Tables

dbo.Authors

dbo.BookCategories

dbo.Books

dbo.Categories

Dropped Ledger Tables

Views

External Resources

Synonyms

Programmability

Query Store

Service Broker

Storage

Security

Server Objects

Replication

Always On High Availability

Management

SQL Server Agent (Agent XPs disabled)

SQLQuery13...emanm (78)*

task4b.sql - ...D\emanm (56)

task4a.sql - ...D\emanm (76)

```
1 USE LibraryDb_Week3;
2 GO
3
4 INSERT INTO BookCategories (BookId, CategoryId)
5 VALUES
6 (1, 1), -- Harry Potter 1 + Fantasy
7 (1, 2), -- Harry Potter 1 + Fiction
8
9 (2, 1), -- Harry Potter 2 + Fantasy
10 (2, 2), -- Harry Potter 2 + Fiction
11
12 (3, 2), -- 1984 + Fiction
13 (3, 4), -- 1984 + Dystopian
14
15 (4, 2), -- Animal Farm + Fiction
16 (4, 4), -- Animal Farm + Dystopian
17
18 (5, 2), -- Pride and Prejudice + Fiction
19 (5, 3), -- Pride and Prejudice + Classic
20
21 (6, 2), -- Sense and Sensibility + Fiction
22 (6, 3), -- Sense and Sensibility + Classic
23 GO
24
```

75 % 4 0 Ln: 25, Ch: 30 TABS CRLF UTF-

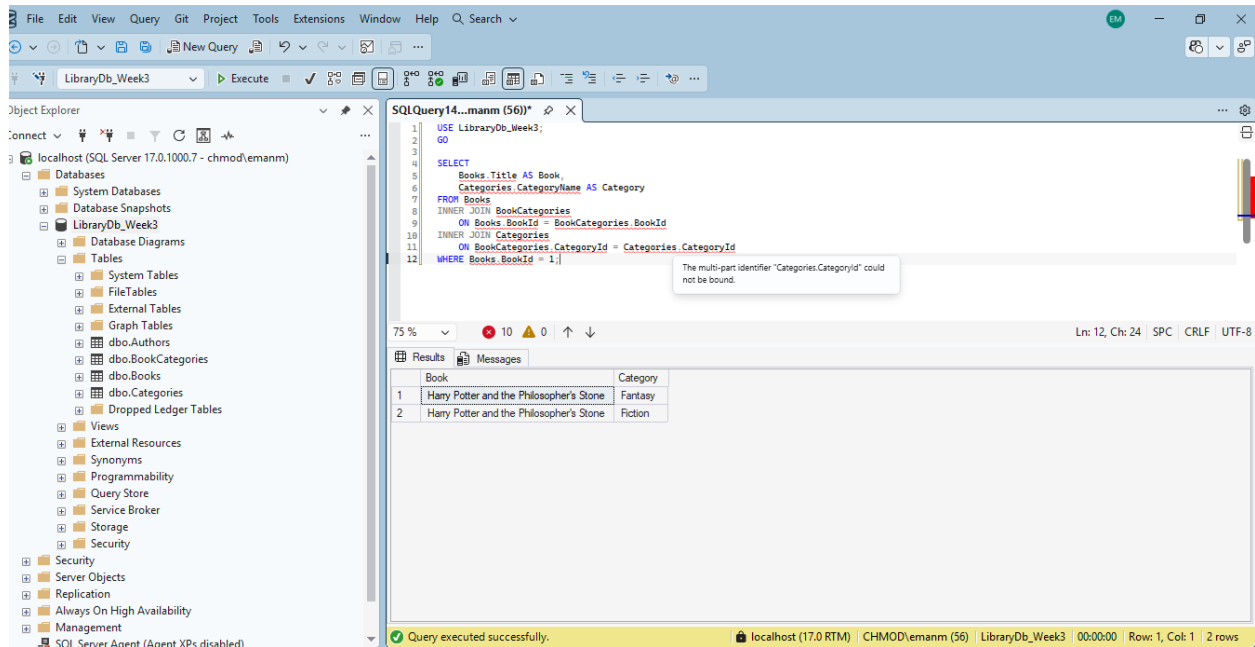
Results Messages

	BookId	CategoryId
1	1	1
2	1	2
3	2	1
4	2	2
5	3	2
6	3	4
7	4	2
8	4	4

Query executed successfully. localhost (17.0 RTM) CHMOD\emanm (78) LibraryDb_Week3 00:00:00 Row: 1, Col: 1 12 rows

5. Write a query that returns all categories for one specific book using the link table.

Output:



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left displays the database structure for 'LibraryDb_Week3'. The central query window shows the following SQL query:

```
USE LibraryDb_Week3;
GO
SELECT
    Books.Title AS Book,
    Categories.CategoryName AS Category
FROM Books
INNER JOIN BookCategories
    ON Books.BookId = BookCategories.BookId
INNER JOIN Categories
    ON BookCategories.CategoryId = Categories.CategoryId
WHERE Books.BookId = 1;
```

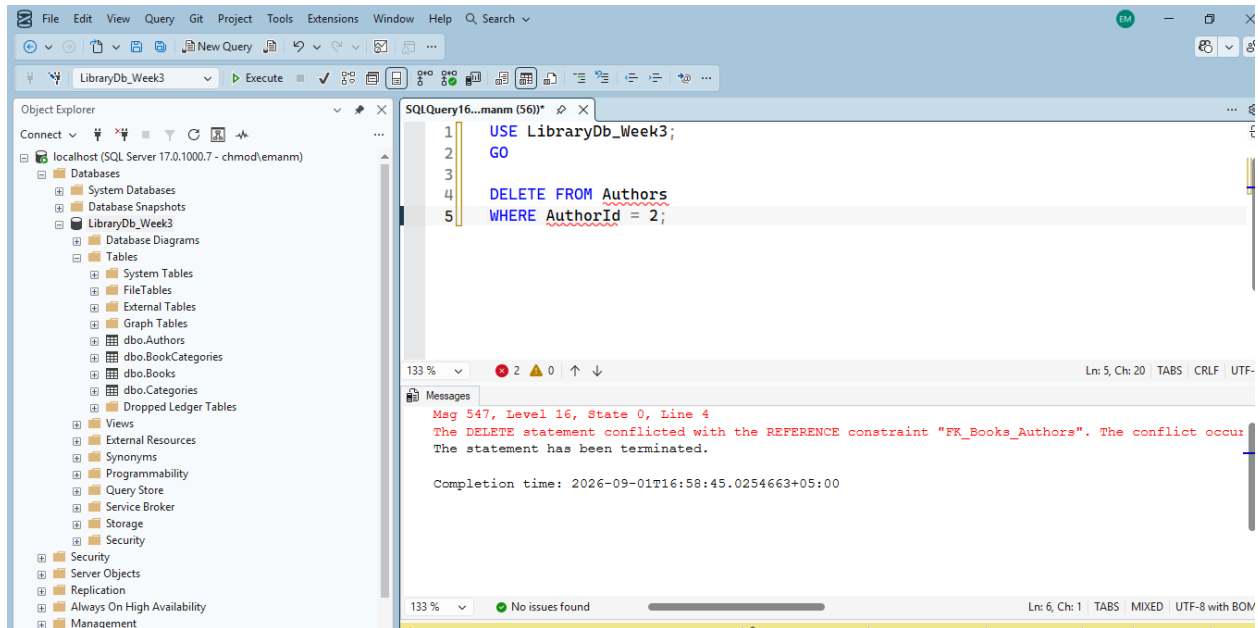
The query results are displayed in a table with two columns: 'Book' and 'Category'. The results show two rows for the book 'Harry Potter and the Philosopher's Stone'.

Book	Category
Harry Potter and the Philosopher's Stone	Fantasy
Harry Potter and the Philosopher's Stone	Fiction

The status bar at the bottom indicates that the query was executed successfully, returning 2 rows.

6. Try deleting an Author that still has Books linked to it and read the error SQL Server gives you — explain in your own words why it refused.

Output:



SQL Server refused to delete the author because the author is still referenced by records in the Books table. The Books table has a foreign key (AuthorId) that references the Authors table. Deleting the author would leave the related books without a valid author, so SQL Server prevented the deletion to maintain referential integrity.